

Volt

Product Requirements Document — Final Internal Working Version

Owner	Nishal
Drafted with	Claude (Cowork)
Date	July 10, 2026
Status	Draft — pre-validation, learn-in-public project. Phase 0 has not started.
Version	1.1 — feature comparison expanded to 8 categories with a Notes column distinguishing deliberate v

What changed in this version: this merges the original PRD draft with the separately-written phase-by-phase build roadmap into one document, and adds a build-in-public content calendar mapped to LinkedIn, X, Reddit, and Show HN / Indie Hackers / Discord for each phase. It also adds an Advisor Notes section flagging where the plan is strongest and where it is still exposed. This is an internal working document — hedging language ([Certain] / [Likely] / [Guessing], "unvalidated," "unconfirmed") is kept intact deliberately, not smoothed over.

Advisor Notes — Read This Before Week 1

This section is not part of the original PRD. It's a direct assessment of the plan's biggest exposure points, added because the single highest-leverage thing you can do right now is not polish this document further — it's start Phase 0.

- **[Certain]** Phase 0 has not started. Every phase after it — including the 8-post content calendar below — assumes momentum that doesn't exist yet. Don't schedule Phase 1 tasks until at least the landing page and one round of outreach (Reddit + 5-10 DMs) are live. A week spent validating is cheaper than 10 weeks spent building something 3 people wanted.
- **[Certain]** The two source documents disagree on timeline: the PRD's Open Questions section says "6-10 week timeline for full P0 scope," while the phase roadmap totals ~10-11 weeks including Phase 0 and hardening. This PRD uses 11 weeks (Phase 0 through Launch) as the working number, since it's the more complete accounting — but budget for this to slip, not compress. Encryption (Phase 5) and MCP (Phase 6) are the roadmap's own flagged risk phases.
- **[Likely]** Google's OAuth verification for sensitive Gmail scopes (readonly + send + modify) is listed as "unresearched" in the PRD. This is worth resolving in Phase 1, not discovering in Phase 9 — if verification requires a review process beyond a small test-user allowlist, it directly affects how Phase 7's "stranger self-hosts it" gate can even work before formal verification is granted.
- **[Guessing]** BYO-OAuth-client-per-self-hoster removes you as a liability/quota bottleneck, which is the right call for a solo maintainer — but it is real onboarding friction stacked on top of Docker Compose setup, encryption setup, and BYO-AI-key setup. Three separate "go get your own credentials" steps in one setup flow is a lot for a non-technical or even mid-level technical audience. Worth timing an honest onboarding run in Phase 7 and being ready to cut scope (e.g., defer AI features to post-connect) if setup time runs past ~15 minutes.
- **[Likely]** "Zero critical security incidents" is listed as non-negotiable, but there's no incident-response plan or disclosure process named anywhere in the doc. For a project handling real Gmail credentials from strangers by Phase 7, a one-paragraph "if something goes wrong, here's what happens" policy costs little and matters if it ever gets used.
- **[Certain]** The content calendar below assumes all four platforms (LinkedIn, X, Reddit, Show HN/IH/Discord) get used at every applicable milestone. Posting on all four for every phase is more volume than most solo builders sustain for 11 weeks. Treat the calendar as a menu, not a mandate — LinkedIn + X consistently, Reddit and Show HN at the milestone moments they're built for (Phase 0 and Phase 8 specifically).
- **[Certain]** The expanded feature comparison (§5) originally labeled the two calendar-related rows "Phase 2" — corrected to "v2" here. Phase 2 in this project's roadmap is Gmail Auth & Core Inbox, and calendar integration is an explicit v1 non-goal (§3); labeling it "Phase 2" would have contradicted that. If calendar work was actually meant to land inside the current v1 roadmap, that's a scope change worth deciding deliberately, not one to let slide in via a table cell.

1. Problem Statement

Developers, self-hosting enthusiasts, and privacy-conscious knowledge workers who want an AI-accelerated, keyboard-first email experience currently have exactly one credible option: Superhuman, closed-source, \$30–40/user/month. No open-source, self-hostable equivalent exists as of July 2026, confirmed via search across multiple "open-source alternative to X" directories.

Evidence status: the absence of a competitor is confirmed. The size of the population that would actually switch is **assumed, not measured** — no interviews, surveys, or waitlist data exist yet. This remains the single largest unvalidated assumption in the project, and it is why Phase 0 exists.

2. Goals and Success Metrics (Learn-in-Public Framing)

This project's primary purpose is skill demonstration and public documentation, not shipping a finished company. Metrics reflect that: completion of every phase is a bonus outcome, not the bar for success.

- **Publish 8+ substantive build-in-public posts** covering real technical decisions and real mistakes, not just progress announcements — this is the primary deliverable.
- **Ship and validate Phase 1 (foundation) within 1 week** — repo, backend/frontend skeleton, Docker Compose, OAuth setup, CI, all working end-to-end.
- **Reach a working Gmail-connected inbox with core keyboard nav (Phases 2–3)** as the minimum bar for a credible portfolio artifact, even if AI/encryption/MCP phases don't get finished.
- **If time allows, complete through Phase 8 (public launch)** — Show HN / Reddit posts, 100 GitHub stars within 30 days of launch, 10 verified self-host installs within 60 days.
- **Zero critical security incidents** in any phase that handles real Gmail credentials — non-negotiable regardless of how much of the roadmap gets built.

3. Non-Goals (v1)

- Outlook/Microsoft Graph support — Gmail only for v1.
- Native desktop or mobile apps — web app only.
- Automatic (non-on-demand) AI drafting or auto-labeling.
- Calendar integration.
- Read receipts / open tracking — deliberate privacy-positioning exclusion, not just a time-saver.
- Team/shared inboxes, comments, CRM integration — enterprise scope, wrong fit for solo v1.
- A hosted/managed Volt cloud service — self-host-only.
- Multi-account support — single Gmail account per instance.

4. Tech Stack

Layer	Choice	Why
Frontend	React / Next.js	Component ecosystem depth, matches most dev-tool job postings, fastest to hire collaborators against if the project grows.
Backend	Go (Gin or Echo)	Goroutines handle concurrent Gmail webhook/sync fan-out more cleanly than Node's event loop or Python's asyncio for this workload; single static binary simplifies self-host deployment.
Database	PostgreSQL via pgx or sqlc	sqlc gives typed, generated queries from raw SQL — avoids ORM overhead, a detail worth explaining in interviews.
Email integration	Gmail API (OAuth2)	Official Go client library exists (google.golang.org/api/gmail/v1); Outlook/Graph deferred to avoid a second integration surface.
Real-time sync	Gmail Pub/Sub push notifications	Polling feels laggy and burns API quota; push is the correct architecture but adds real setup complexity — budget time for this.
AI layer	Claude / OpenAI API, BYO key	User supplies their own key — no inference cost absorbed by the maintainer, and prompts are visible/auditable in the repo rather than a black box.
Encryption	age or libsodium (client-side)	Zero-knowledge pattern — same approach Bitwarden uses for credential sync; server only ever stores ciphertext.
Deployment	Docker Compose	Single-command self-host is the core differentiator from Superhuman; must be low-friction or the differentiator is theoretical.
Agent integration	Native MCP server	Exposes inbox read/search/draft as tools Claude/Cursor can call — the project's clearest, most current skill signal.

5. Feature Comparison: Volt vs. Superhuman

Full comparison, organized by category. The Notes column flags where Volt's "No" is a deliberate scope choice for v1 versus a genuine longer-term gap.

5.1 Core Email Experience

Feature	Superhuman	Volt v1	Notes
Keyboard-first navigation	Yes — 100+ shortcuts	Partial — ~15–20 core shortcuts	Expandable over time
Command Palette (Cmd+K)	Yes	Yes	Cheap to build, high impact
Split Inbox (AI-categorized)	Yes — 5 auto-categories	Partial — manual rule-based only	AI auto-categorization is v2
Snippets/templates with variables	Yes	Yes	
Send later / scheduling	Yes — AI-predicted optimal time	Partial — manual time picker only	
Undo send	Yes	Yes	Trivial to implement
Multiple account support	Yes	Partial — v2	Single account for v1
Native desktop app	Yes	No	Web app only, v1
Native mobile app	Yes — iOS/Android	No	Not planned near-term

5.2 AI Features

Feature	Superhuman	Volt v1	Notes
AI Auto-Summarize	Yes — live-updating	Yes — on-open, cached	
AI Auto Draft (voice-matched, automatic)	Yes	Partial — on-demand draft only	Automatic pre-drafting is v2
Ask AI (natural language search)	Yes — inbox+calendar+web	Partial — inbox only	
Auto-archive via AI prompts	Yes	No — v1	Needs a trained triage layer
Follow-up detection + auto-draft	Yes	No — v1	Manual snooze only
AI model transparency	No — fixed, undisclosed pipeline	Yes — visible prompts, swappable model	Volt advantage
Bring-your-own AI key	No	Yes	Volt advantage
Local/offline AI model support	No	Partial — roadmap (Ollama-style)	Volt advantage, once built

5.3 Calendar & Scheduling

Feature	Superhuman	Volt v1	Notes
Inline calendar availability	Yes	No	v2 — real scope, own API surface
Create events from email via AI	Yes	No	v2

5.4 Collaboration / Team

Feature	Superhuman	Volt v1	Notes
Shared/team inbox	Yes — Business tier	No	Out of scope — enterprise feature, wrong fit for solo OSS v1
Team comments on threads	Yes — Business tier	No	Out of scope
CRM integration (HubSpot/Salesforce)	Yes — Business tier	No	Out of scope

5.5 Provider & Platform Support

Feature	Superhuman	Volt v1	Notes
Gmail support	Yes	Yes	
Outlook/Microsoft support	Yes	No	v2 — second integration surface
Other IMAP providers	No	Partial — roadmap	Potential Volt advantage if built

5.6 Security, Privacy & Data Ownership

Feature	Superhuman	Volt v1	Notes
Open-source, auditable code	No	Yes	Volt advantage
Self-hostable	No — SaaS-only	Yes	Volt advantage
Zero-knowledge client-side encryption	No — undisclosed	Yes	Volt advantage
Data residency control	No	Yes — self-host = your choice	Volt advantage
Read receipts (tracking pixels)	Yes	No — deliberately excluded	Privacy-first choice, also avoids infra cost
Transparent telemetry	No	Yes — nothing hidden	Volt advantage

5.7 Pricing & Business Model

Feature	Superhuman	Volt v1	Notes
Individual plan	\$30/user/month	Free (self-host)	You pay only your own AI API usage
Business/AI tier	\$40/user/month	Free	
Vendor lock-in risk	Yes — real risk if pricing changes/shuts down	No — code is permanently forkable	Volt advantage

5.8 Openness & Community

Feature	Superhuman	Volt v1	Notes
Forkable / customizable	No	Yes	Volt advantage
Community contributions possible	No	Yes	Volt advantage
Native, open MCP server	Partial — bolted-on, closed	Yes — first-class, open	Volt advantage — your headline feature
Public roadmap	No	Yes, if run that way	Your choice, but a strong community-trust signal

The one-line honest summary for the README: Volt loses on raw feature count and AI polish (\$5.1–\$5.4), and wins decisively on ownership, transparency, and cost (\$5.6–\$5.8). That's the actual pitch — don't try to hide the gaps in the top half, since any developer who tries it will find them in five minutes. Lead with the bottom half.

6. User Personas

Primary — Priya, backend engineer

Pays for Superhuman out of pocket, likes the speed, uneasy that a third party has standing access to her inbox, watched Notion/Slack bundle AI into price hikes in 2026 and doesn't want the same surprise from Superhuman. Technical enough to self-host on a small VPS. Finds Volt via a Show HN post, self-hosts over a weekend, checks the AI prompt source before trusting it with real email.

Secondary — Daniel, engineering manager

12-person team, internal policy against sending customer email content to opaque third-party AI. Superhuman is a non-starter regardless of budget. Interested in client-side encryption and BYO-model support. Higher trust bar than Priya — better thought of as a v2+ target once Volt has a track record.

7. User Stories and Requirements

P0 — Must-have

User Story	Rationale
Connect Gmail via OAuth	Core dependency for everything else.
Deploy via Docker Compose	Self-hosting is the core differentiator; painful setup makes it theoretical.
Keyboard-navigate the inbox	Half of Superhuman's value prop is speed, not just AI.
Command Palette (Cmd/Ctrl+K)	Cheap to build, disproportionate impact on "feels fast" perception.
AI Summarize on demand	Highest-value, lowest-complexity AI feature; good demo material.
AI Draft reply on demand	On-demand (not automatic) reduces risk of visibly wrong AI behavior on a real inbox.
Client-side encrypted credential storage	Structural differentiator claim must actually be true, not just marketed.
Native MCP server	Headline differentiator; most direct signal of the agentic-AI skill being demonstrated.

P1 — Should-have

User Story	Rationale
Snippets/templates with variables	Low cost, no new integration surface.
Scheduled send (manual time)	Simple; AI-predicted timing deferred to v2.
Manual rule-based inbox splitting	Partial Split Inbox value without triage-model risk.
Natural language inbox search	High perceived value; scoped to inbox-only to bound effort.

P2 — Deferred past v1

User Story	Rationale
Automatic AI inbox categorization	Needs a trained triage layer; real risk of visibly wrong behavior.
Automatic follow-up detection	Polish feature, not core to the value proposition.
Outlook support	Second OAuth/API surface — deliberately deferred.
Native desktop app	Web app sufficient to validate the core hypothesis first.
Inline calendar availability / create events via AI	Surfaced by the \$5 feature comparison — real scope, own API surface. Not previously tracked; added here for consistency.
Local/offline AI model support (Ollama-style)	Surfaced by the feature comparison as a roadmap idea, not a committed scope item yet. Genuine Volt advantage if built — revisit after v1 ships.
Other IMAP providers (beyond Gmail)	Same status as above — a real potential differentiator, currently just an idea with no committed timeline.

8. Non-Functional Requirements

Security

- OAuth tokens encrypted at rest at minimum, client-side encryption preferred (P0).
- AI API keys stored encrypted, never logged, never included in error reports.
- Minimum-necessary Gmail scopes only — reduces risk and OAuth-verification burden.

Performance

- Inbox list interactive within 2 seconds for a 90th-percentile inbox — exact target size is an assumption, should be validated with early self-hosters.
- AI actions show a loading state within 300ms of click.

Compliance / Accessibility — explicitly deferred

No WCAG audit, no formal compliance work, no penetration test planned for v1 given the solo, no-budget constraint. Team-scale usage (Daniel's persona) should not be represented as compliance-ready until this work is actually done.

Scalability — explicitly deferred

V1 targets single-user, single-instance self-hosting only. No multi-tenant architecture planned.

9. Primary User Flow

- Clone repo / pull Docker image, run `docker compose up`.
- Create a Google Cloud OAuth client (self-documented step-by-step).
- Paste OAuth client ID/secret into Volt's setup screen.
- Click "Connect Gmail," complete OAuth consent with minimum scopes.
- Volt performs initial sync, displays inbox.
- Optionally add an AI provider key to enable summarize/draft; inbox/keyboard-nav works without it.

Edge cases

- OAuth consent denied/fails — clear retry path, no partial state.
- Gmail API rate limit during sync — graceful degrade (partial sync + background retry).
- Very large inbox (100k+ messages) — sync strategy not yet decided, open question.
- Invalid AI key or provider rate limit — visible error, never a silent or fabricated response.
- Externally revoked Gmail access — detected on next sync, prompts reconnection.

10. Technical Risks

- **[Inference]** Real-time feel likely requires Gmail Pub/Sub push, not polling — not yet confirmed against an actual API rate-limit budget.
- **[Inference]** Google's OAuth verification for sensitive scopes likely requires app review beyond a small test-user allowlist — not researched in depth, meaningful unknown for onboarding smoothness past early self-hosters.
- **[Founder-confirmed, not engineer-validated]** Each self-hoster provides their own OAuth client rather than Volt shipping a shared one — avoids the maintainer being a single point of quota/liability, but adds real setup friction.
- **[Inference]** Client-side encryption adds real engineering time beyond basic CRUD sync — budget for it explicitly.
- **[Unconfirmed]** AI quality with a general-purpose model vs. Superhuman's purpose-built pipeline has not been prototyped or benchmarked.

11. Open Questions and Risks

- Unvalidated: real switching demand for "open + self-hosted" separate from AI quality — no waitlist/survey/interview data exists yet.
- Unvalidated: solo-builder capacity to maintain security patches and support once real users exist.
- Unresearched: whether Google's OAuth verification becomes a hard onboarding blocker at scale.
- Reasonable but unconfirmed: BYO-AI-key architecture is an acceptable tradeoff for target users.
- Founder estimate, not engineering-validated: 6–10 week timeline for full P0 scope — encryption and MCP server carry the most risk of running long.
- Unreviewed: legal/ToS risk of operating a third-party Gmail client at scale.

12. Phase Roadmap — Detailed

Total timeline: ~10–11 weeks, solo, part-time-to-full-time. If building nights/weekends around a job, double the calendar — don't compress the phases, extend the schedule instead.

Phase 0 — Validation

Week 1 — Find out if anyone other than you wants this before spending 10 weeks building it. The highest-leverage week of the whole roadmap.

- Write a one-page landing page: what Volt is, why it's different, a waitlist field or GitHub watch/star CTA
- Post it in *r/selfhosted*, *r/opensource*, *r/programming*, and relevant dev Discords/Slacks
- Post a "here's what I'm about to build and why" thread on LinkedIn/X
- DM 5–10 developers who use Superhuman or complain about SaaS AI pricing
- Track: waitlist signups, GitHub stars/watches, comment sentiment

Exit gate: At least some signal that real people want this — not zero response. Dead silence everywhere means dig into why or reconsider scope/positioning before committing 10 more weeks.

Phase 1 — Foundation & Setup

Weeks 1–2 — Boring infrastructure, done right once so you don't fight it later.

- Repo setup: monorepo or split frontend/backend, license (MIT or Apache 2.0)
- Go backend skeleton (Gin/Echo), Postgres connection (pgx or sqlc), health-check endpoint
- Next.js frontend skeleton, deployed to a dev environment
- Docker Compose file spinning up backend + Postgres + frontend together
- Google Cloud OAuth client setup + self-hosted documentation
- CI: basic lint/build check on push

Exit gate: docker compose up on a clean machine gives a running (empty) app with a health-check passing.

Phase 2 — Gmail Auth & Core Inbox

Weeks 2–4 — The unglamorous, load-bearing feature. Nothing else works without this.

- Google OAuth2 consent flow, minimum-necessary scopes (readonly + send + modify labels)
- Store tokens encrypted, never logged
- Initial inbox sync (recent-N-messages-first recommended over full history)
- Inbox list view: subject, sender, snippet, read/unread state
- Open/read a full email thread; basic actions: archive, reply, mark read/unread
- Handle error states: OAuth failure, revoked token, rate limits

Exit gate: Connect your own test Gmail account, see your inbox, read an email, archive it — end to end, no AI yet.

Phase 3 — Speed Layer

Week 4 — Half of Superhuman's value proposition. Build it before AI so the product feels fast even with zero AI calls.

- Core keyboard shortcuts: navigate list (j/k), open/close, archive, reply, compose, search focus (~15–20 shortcuts)
- Command Palette (Cmd/Ctrl+K) wired to every action built so far
- Basic snippets/templates with variable substitution

Exit gate: You can operate the entire inbox without touching the mouse.

Phase 4 — AI Layer

Weeks 5–6 — The features that make this an AI-native client, not just a fast one.

- BYO-AI-key settings (Claude/OpenAI API key, stored encrypted)
- AI Summarize: one API call per thread, cache the result, display above the thread
- AI Draft (on-demand): user clicks "draft reply," AI generates, user edits before sending — never auto-sends
- Natural language inbox search (inbox-only, not calendar/web)
- Prompt templates kept in a visible, documented file in the repo

Exit gate: Summarize and draft both work reliably on real threads; search returns relevant results for test queries.

Phase 5 — Security Layer: Client-Side Encryption

Week 7 — The structural claim ("self-hosted server never sees plaintext") has to actually be true, not just marketed.

- Client-side encryption for stored credentials/tokens (age or libsodium)
- Verify server-side storage is genuinely ciphertext-only — test this explicitly, don't assume
- Document the encryption approach in the repo README security section

Exit gate: You can inspect your own Postgres database directly and confirm no plaintext credentials are visible.

Phase 6 — MCP Server

Week 8 — Your headline differentiator and the most direct signal of the exact skill you're trying to demonstrate to employers.

- Build an MCP server exposing inbox read/search/draft actions as tools
- Test it end-to-end with Claude Code or Cursor actually calling it
- Document setup so others can connect their own Claude/Cursor to their self-hosted Volt instance

Exit gate: You can ask Claude (via MCP) to "summarize my last 5 unread emails" and get a real answer from your live inbox.

Phase 7 — Self-Host Hardening & Real User Test

Week 9 — Your own dev machine is not a representative test of onboarding. Fix that before launch, not after.

- Clean-machine test: wipe a VM, run only the public setup instructions, note every friction point
- Recruit at least one person who isn't you to self-host it from scratch, watch where they get stuck
- Fix the top 3–5 friction points found
- Write the actual README/setup docs a stranger needs — assume zero prior context

Exit gate: Someone who is not you has successfully self-hosted Volt and connected their own Gmail account without your direct help.

Phase 8 — Public Launch

Week 10 — Get it in front of the audience validated in Phase 0.

- Finalize README: what it is, why it exists, screenshots/GIF, quickstart, security section, roadmap link
- Show HN post
- Product Hunt listing (optional, lower priority than Show HN/Reddit for this audience)
- r/selfhosted, r/opensource launch posts
- LinkedIn/X launch thread tying together the whole build-in-public arc
- DM the people who showed interest in Phase 0 — tell them it's live

Exit gate: Public repo, public post, real traffic. This is a milestone, not an ending.

Phase 9 — Post-Launch

Week 11+, ongoing — A live launch is not the finish line for either the community goal or the job-market goal — sustained activity is what actually gets noticed.

- Respond to every GitHub issue within 48 hours for the first month
- Ship P1 features (scheduling, manual split inbox, snippets polish) based on real feedback, not the original guess
- Weekly-ish "what I shipped this week" posts — consistency beats intensity for long-term reach
- Reach out to Superhuman directly once the project has a real track record, not on day one

Exit gate: N/A — this phase doesn't end; it's the maintenance mode that determines whether Volt becomes a real credential or a repo that goes quiet after week 12.

Summary Timeline

Week	Phase	Key Output
1	Validation	Landing page, real signal on demand
1–2	Foundation	Working Docker Compose skeleton
2–4	Core Inbox	Gmail auth + read/archive/reply working
4	Speed Layer	Full keyboard nav + Command Palette
5–6	AI Layer	Summarize + on-demand draft + search
7	Encryption	Verified client-side encrypted storage
8	MCP Server	Claude/Cursor can query your live inbox
9	Hardening	A stranger self-hosted it successfully
10	Launch	Public repo, Show HN, launch thread
11+	Post-launch	Issue response, P1 features, weekly posts

The one rule that matters most across all of this: don't skip Phase 0, and don't skip Phase 7. Every other phase is you building; those two are the only phases where you find out if you're building the right thing and whether a stranger can actually use it. Skipping either is the most common reason build-in-public projects launch to silence.

13. Definition of Done (v1)

- All P0 stories shipped and working end-to-end on a real Gmail account.
- Docker Compose self-host path tested by someone other than the builder.
- Client-side encryption actually implemented and independently verified, not silently deferred.
- At least 8 build-in-public posts published documenting real decisions, including at least one post about something that broke or went wrong.

Section 14 (Build-in-Public Content Calendar) continues on the following landscape pages.

14. Build-in-Public Content Calendar

Mapped to the phase roadmap above. Treat this as a menu, not a mandate (see Advisor Notes) — LinkedIn and X sustained weekly, Reddit and Show HN/Indie Hackers/Discord concentrated at the milestone moments they suit best (Phase 0 validation and Phase 8 launch, plus the Phase 6 MCP demo, which is genuinely novel enough for a second Show HN-adjacent moment).

Phase	LinkedIn	X (Twitter)	Reddit	Show HN / IH / Discord
Phase 0 — Validation (Week 1)	Origin-story thread: "I'm about to spend 10 weeks building an open-source Superhuman alternative. Here's the gap and why it's worth building." Ask readers to watch/star the empty repo.	Same story, shorter thread (5–7 tweets), tag #buildingpublic / #opensource / #indiehackers. Pin it — this is the post people reference later.	r/selfhosted, r/opensource, r/programming: share the landing page directly, ask for honest "does this already exist" feedback, not upvotes.	Indie Hackers: intro/milestone post. Relevant dev Discords/Slacks: DM 5–10 people directly, don't just broadcast.
Phase 1 — Foundation (Weeks 1–2)	"Day 1 of building in public: repo is live, here's the stack and why I picked Go over Node for the backend." Link the repo.	Same post, code screenshot or terminal GIF of docker compose up succeeding.	Skip — save Reddit for milestones, not setup updates. Comment in the original Phase 0 thread instead.	Post the CI-passing badge / repo link in build-in-public accountability Discords.
Phase 2 — Core Inbox (Weeks 2–4)	Short screen recording: "Volt can now read and act on a real Gmail inbox. No AI yet — making sure the foundation is solid first."	Same clip, framed as unglamorous-but-real progress — this format under-performs on X unless the visual is tight, so trim to <20s.	Skip unless a real technical decision worth debating comes up (e.g., recent-first vs. full-history sync).	Progress check-in in the same channels as Phase 1.
Phase 3 — Speed Layer (Week 4)	GIF of flying through the inbox keyboard-only, framed around Superhuman's core value prop.	Same GIF — this is historically the best-performing format on X, speed is visually obvious even to non-technical viewers.	r/selfhosted likely receptive to a speed demo tied to self-hosting angle.	Share in Indie Hackers as a milestone update.
Phase 4 — AI Layer (Weeks 5–6)	Side-by-side demo: "Here's Volt summarizing a long thread in one click — and here's exactly what prompt it sent." Link the prompt file in the repo.	Thread version with the prompt screenshot inline — transparency-as-a-feature is the strongest differentiation story, use it directly.	r/opensource: prompt-transparency angle specifically resonates here.	AI/agent-focused Discords: post the prompt file link, invite scrutiny rather than just praise.
Phase 5 — Encryption (Week 7)	Technical deep-dive post: "How Volt keeps your credentials encrypted even from the self-hosted server — same pattern Bitwarden uses." Most resume-relevant post — don't skip writing it.	Condensed thread version linking to the full write-up (repo README or a blog/Substack post).	r/selfhosted and security-adjacent subs: technical audience will scrutinize this one — be ready to defend the specifics.	Security-focused Discords/communities where the zero-knowledge pattern will get real technical pushback (useful signal).
Phase 6 — MCP Server (Week 8)	"I connected Claude to my own open-source email client via MCP — here's what that looks like." Best launch-adjacent post; high novelty.	Demo video/GIF, tag relevant agentic-AI conversation and MCP-focused accounts/hashtags.	r/programming, MCP- and LLM-tooling-focused subreddits.	MCP/Claude builder Discords — this is the single most relevant community for this specific milestone.

Phase	LinkedIn	X (Twitter)	Reddit	Show HN / IH / Discord
Phase 7 — Hardening (Week 9)	Honest post: "I had someone else try to self-host Volt from scratch — here's everything that broke." Vulnerable, builds more trust than polished demos.	Same framing, thread format — this style of post typically outperforms polished demos on engagement.	r/selfhosted: friction-focused audience will appreciate the honesty and may offer fixes.	Share in the same Discords/IH thread as a credibility-building update ahead of launch.
Phase 8 — Public Launch (Week 10)	Capstone post: "10 weeks ago I found a gap... Today Volt is live." Link everything — repo, Phase 0 post, security deep-dive, MCP demo.	Launch thread tying the full build-in-public arc together — this is the post to link in job applications.	r/selfhosted, r/opensource launch posts (separate, tailored per-sub, not identical copy-paste).	Show HN post (primary launch channel for this audience); Indie Hackers milestone; DM everyone who showed interest in Phase 0.
Phase 9 — Post-Launch (Week 11+, ongoing)	Weekly "what I shipped this week" posts — consistency beats intensity for long-term reach.	Same cadence, shorter form; respond to every reply/quote-tweet promptly.	Occasional updates tied to real feature releases, not a fixed schedule — avoid oversaturating.	Ongoing GitHub issue responses (48hr SLA for month 1); periodic IH/Discord check-ins.